Analysis of Randomised Algorithms in Lean 4

Antoine du Fresne von Hohenesche

Master's thesis, Department of Mathematics, ETH Zürich
Supervisor: Sorrachai Yingchareonthawornchai
Co-supervisor: Georg Anegg

23 August 2026

Abstract

A randomised algorithm is at once a program that runs, a distribution over its possible outputs, and a random cost. Proof assistants have been used to analyse such algorithms before, mostly in Isabelle and Coq, and the approaches differ in how much of the algorithm is turned into data that the assistant manipulates and how much stays an ordinary function. This thesis presents ARA (Analysis of Randomised Algorithms), a framework in the Lean 4 proof assistant in which an algorithm is written once, as one ordinary function, and that single text is then run, analysed for correctness and analysed for cost. The cost model is written into the algorithm itself, next to the operations it charges. The thesis explains the framework and its design. It describes automation that, we hope, spares the user the non-mathematical work of the analysis of some randomised algorithms. Nine formalised algorithms are reported as applications. Among them is Karger's contraction algorithm, which on a multigraph $G = (V, E)$ returns a minimum cut with probability at least $2/(|V|(|V| - 1))$; to the best of our knowledge, this algorithm had not been formally analysed before. The code described here is at commit `765761e` of the repository `github.com/AntoineduFresne/Master-Thesis`; the repository will continue to change after this thesis.

Acknowledgements

I thank Sorrachai Yingchareonthawornchai for proposing this subject and for the discussions that shaped the design of the framework. I also thank Georg Anegg for his advice on making this thesis and his availability throughout.

1 Introduction

Consider a sentence such as: "Quicksort makes $2(n + 1)H_n - 4n$ expected comparisons on a list of n distinct entries". Three objects hide behind the one word Quicksort. There is a program, which someone runs. There is a distribution over the possible outputs, since the pivots are drawn at random (and the correctness of Quicksort is a statement about that distribution: it is the Dirac distribution concentrated on the sorted permutation of the input list). And there is a cost of the algorithm (the time complexity of it), here the number of comparisons. The cost is random as well, and the sentence reports its expectation.

In Lean 4 [dMU21] the three are terms of three different types. The program reads a pseudorandom generator of the machine to draw the pivot, and in Lean a function that reads such a source must announce it in its type, so its type is not that of an ordinary function from lists to lists. The distribution is a probability mass function over lists. The cost, and its expectation, is a real number. With these differences of type, one common practice for analysing such an algorithm is to define it multiple times, once for each analysis purpose: as a function into the type of distributions (of the output), to analyse the output; as a recurrence, or again as a function into the type of distributions (of the cost), for the expected cost or to analyse the cost; and as a function using a pseudorandom

number generator, to run it. The three definitions then have to be certified as talking about the same algorithm, either by reading them, which one will have to trust, or by proving bridge theorems between them, which is more work and has to be maintained after every edit of one of the representations. We were not comfortable with the drift this allows. Section 2 places this practice among the deep, shallow and hybrid approaches and gives our reasons in full. The question we set ourselves was whether a single Lean definition can play all three roles at once, and what a reader has to trust for that: we wanted that trust to be small, and to be stated. Throughout this document, many notions live on the programming side of Lean 4; for convenience, we assume that the reader is familiar with the book *Functional Programming in Lean* [Chr23]. Still, Section 3 adds, and re-explains, the few further notions that the thesis sections use.

ARA, our framework, implements a randomised algorithm as a shallow embedding in the sense of Section 2: an algorithm is an ordinary Lean function written in do-notation. An ARA algorithm is also polymorphic over a monad M that carries what we call a "source of randomness", a primitive draw; such an M is a "random monad". In Section 2 we report some Isabelle work that writes each algorithm as a function returning a distribution, so that its monad is fixed. We, instead, made the monad a parameter, since that seemed to us the simplest way to keep one definition for every way of reading the algorithm. To the best of our knowledge, no Lean development is structured this way, and Section 2 names the closest work in other systems. Two small typeclasses connect the text of the algorithm to its analyses, `RandMonad` and `MonadCost`. `RandMonad` provides the primitive function `randFin n`, which will be interpreted as "drawing" among the integers $0, \ldots, n-1$. For charging a cost to a step, `MonadCost` provides what we call `tick`, which is a syntactic marker that adds a stated cost to the current branch of the algorithm. Instantiating M at some monads that implement these two typeclasses then gives to the same text distinct readings. At the monad `IO`, the text is an executable program (why one would want to execute an algorithm in our framework, when the goal is to analyse it, is a question we answer in Section 3). At the monad `PMF`, Mathlib's type of probability mass functions [mat20], it is the distribution of the output. We also build, on `TimeM` from the CSLib library [BCM+26], a cost-counting monad transformer `TimeMT`. Such a transformer, applied to `IO` or to `PMF`, gives respectively a run paired with its cost and the joint distribution of the output and the cost. The theorems of our algorithms are stated generically in our code, over any random monad M that satisfies some further conditions, those that let us prove statements about the algorithm read at M. Section 4 states these conditions.

These are the main design decisions of the framework: the algorithm is written once, polymorphic over its monad, with its cost model written syntactically into it. The rest of the document is organised around the following three points, and says where each of them is developed.

The first point is that our framework seems to be enough for the correctness and the cost analysis of textbook randomised algorithms. Section 4 presents the framework in detail: the two typeclasses, and how one definition of an algorithm, written once, is read in the four ways stated before. Section 8 presents our evidence: nine case studies, from Quicksort and reservoir sampling to Karger [Kar93]. Section 5 explains that the framework adds one thing to trust beyond the Lean kernel. Section 9 says what the framework cannot yet do and states future directions of work.

The second is a classification. While writing the case studies we found ourselves stating the correctness of an algorithm in one of three ways, and we came to call them tiers; two of them have the classical names Las Vegas and Monte Carlo. Section 6 defines the three tiers and, next to them, three cost statements: the distribution of the cost, its expectation, and a tail bound.

The third is about how much of the work the framework does. For Las Vegas algorithms (correct on every run) and for the expected cost, it appears to us that, once the numbered steps are followed, little beyond the mathematics of the algorithm is left to the user; this claim will have to stand the test of time against more complex Las Vegas algorithms than ours, which we did not attempt. Section 6 says what the framework automates and where the automation stops; Section 7 walks the toy algorithm `RandMax` through the numbered steps and shows at each step what the user supplies and what the framework discharges.

A reader who knows well Lean and the literature on formalised probability can go straight to Sec-

tion 4: Sections 2 and 3 place this work among the existing approaches and recall the notions the later sections use. The framework itself is Sections 4 to 7; Section 8 reports what we formalised with it, and Sections 9 and 10 state its limits and our use of AI assistants.

2 Approaches to formalising randomised algorithms

Formal reasoning about randomised algorithms spans a spectrum, and the position a framework takes on it largely decides two things: what its theorems can say about the cost of an algorithm, and how much of the mathematics already in the proof assistant's library is available for proving them. We describe the three usual positions and then place ARA. We have not built frameworks in the other styles, so where we compare, we repeat the accounts that the literature gives of itself rather than our own measurements.

At one end sit deep embeddings. There the algorithm is a term of an inductive type whose constructors are the constructs of a small probabilistic language, such as drawing a random number, sequencing two steps, branching and looping. A separate interpreter, the operational semantics of that language, says what each term does and what each step costs. The advantage is that the cost of the algorithm is computed by the semantics, not declared by the author of the algorithm. A cost theorem is thus a theorem about the semantics, and a reader has to trust nothing beyond the definition of that semantics. A deep embedding also handles imperative programs directly, since an assignment can be a constructor. The disadvantage is that every algorithm and proof works through the small language. A program can call a function of the proof assistant's library, recurse or use dependent types only if some constructor provides that construct. Whatever the language lacks must be rebuilt from its constructors, so such languages tend to stay small.

At the other end sit shallow embeddings. There the algorithm is an ordinary function of the proof assistant's own language (in Lean its dependent type theory) returning a distribution. Some authors call this way of working "modelling directly in the logic". The deterministic steps of the algorithm, such as comparing two elements or splitting a list, are ordinary functions, while a random draw is a call to a primitive distribution such as the uniform distribution on n elements. The type of distributions is a monad whose `bind` updates the distribution built so far by a draw from it, so a `do` block over that type starts from one distribution, each further line updates it, and the whole block is the distribution of the algorithm's output; Section 3 states this monad. Petcher and Morrisett take this approach in Coq [PM15], as does Hölzl in Isabelle [Höl17], and Eberl, Haslbeck and Nipkow analyse randomised Quicksort and random search trees in Isabelle as functions returning a distribution [EHN20]. In each work the algorithm has one fixed type, so its monad is fixed rather than abstracted, as ours will be. The advantage of a shallow embedding is the full expressiveness of the proof assistant's language, with the whole of the proof assistant's library available in proofs. The disadvantage is that nothing connects the code of the algorithm to a cost model; instead, a cost function defined beside the code is matched to it by reading, not by the kernel. Take Quicksort: in this style its output is a function from lists to distributions over lists, and the correctness theorem is about that function. Its expected number of comparisons on a list of n distinct entries is a second definition, a recurrence written by hand from a reading of the code: $T(n) = (n-1) + \frac{1}{n}\sum_{i<n}\big(T(i) + T(n-1-i)\big)$, where $n-1$ is the author's count of the comparisons in the partition step and the average over i is the author's transcription of the uniform pivot draw and the two recursive calls. The theorem $T(n) = 2(n+1)H_n - 4n$ is a theorem about T, and the kernel checks it. However, no term of the proof assistant relates T to the function that defines the algorithm. The two definitions can therefore drift apart. If one changes the partition step of the function, the theorem about T still typechecks, but it now describes an algorithm other than the one written, and nothing in the proof assistant notices.

In the middle sit hybrid embeddings. As the name suggests, it is a mix of a deep and a shallow embedding. The algorithm exists twice, as a syntax tree in a small embedded language and as a distribution defined in the proof assistant's language, with an adequacy theorem tying the two together; the work of Tassarotti and Harper is of this kind [TH19]. The deep side, as one would

expect, carries the operational semantics, so the cost of a program is read off its syntax. The shallow side is an ordinary function of the proof assistant's language (as in the previous paragraph), so the proof assistant's libraries are available to it, and it carries the correctness proofs. The disadvantage of such an embedding is duplication: every construct exists twice, and the full expressiveness of the proof assistant's language is available on the shallow side only. The adequacy theorem does answer the drift of the previous paragraph, since the kernel now checks that the two texts agree; but it is work, and it has to be maintained after every edit of either text. One could reply that lines of code are cheap today and that maintaining this bridge theorem is affordable. In a sense it is; but there are then two texts to keep in step, and an assistant, human or automated, that edits one text has a shorter loop than one that edits two. If the goal is one algorithm with several readings, it seemed to us better to obtain the readings by design, from a single definition, than to reconstruct them afterwards with a bridge theorem. The next paragraph describes how we realised that choice.

ARA is a shallow embedding in the sense described above. What is embedded is the pseudocode of textbook randomised algorithms: its deterministic steps, sequencing, branching and recursion are Lean's own, and the two constructs that remain, the random draw and the charging of a cost, enter as operations of two small typeclasses (Section 4), not as constructors of an inductive type. One change narrows the disadvantage of a shallow embedding: the cost model is written back into the same text as tick calls. A tick call `tick c` stands in the text of the algorithm next to an operation and charges c units for it. The recurrence for the expected cost is then no longer written by the author from a reading of the code. Instead the framework derives it from the text itself, so it cannot drift from the algorithm. What remains to be trusted, and Section 5 states it exactly, is what each tick charges and where it stands. CSLib makes the same trade-off with its cost monad `TimeM` [BCM+26], whose elements are pairs of a result and an accumulated cost; since we build on that file, we inherit its assumption.

What is new here, on that spectrum, is the following. Lean's Mathlib supplies the distributions and CSLib supplies a cost monad; as far as we know, nothing had yet combined the two into a framework for analysing randomised algorithms, and ARA does. One part of this design has a precedent: the Monae library in Rocq [ANS21] writes programs against an interface that only names the operations of a monad, states separately the equations they must satisfy, and proves theorems from those equations alone, so that they hold of every monad satisfying them; its interfaces include a probability monad. Our `RandMonad` and its lawful companion (Section 4) follow the same arrangement; we did not compare the two in detail.

Finally, we acknowledge Lean work that is shallow on this spectrum but goes the other way on the cost side, towards a machine-checked program semantics. The Loom and LeanCP projects formalise an executable semantics in Lean, under which a cost model could be checked rather than trusted. Section 9 reports them as a way our trust assumption could eventually be removed.

3 Background

We recall that we assume of the reader what *Functional Programming in Lean* covers, especially: monads with `pure` and `bind`, `do`-notation as sugar for chains of `bind`, typeclasses and instances, and monad transformers. This section recalls how a `do` block desugars and adds and explains five more notions, all coming from Lean itself or from Mathlib.

Operations versus equations. Lean separates two classes. `Monad` M provides the two operations `pure` and `bind`. `LawfulMonad` M guarantees three equations about them: for all types α, β, γ and all $a : \alpha$, $m : M\,\alpha$, $f : \alpha \to M\,\beta$ and $g : \beta \to M\,\gamma$, one has

```
pure a >>= f = f a                          (left identity)
m >>= pure = m                              (right identity)
(m >>= f) >>= g = m >>= fun a => f a >>= g  (associativity)
```

The distinction matters as follows. Running a program needs only the operations: the compiler executes `pure` and `bind` as defined and never asks whether they satisfy the equations. However, proving something about a program can need additionally the equations: the proof rewrites the program with them, for instance collapsing `pure a >>= f` into `f a`. A theorem about a program therefore assumes `LawfulMonad` M; running the program does not. We keep this separation between operations and equations in ARA for every class we introduce: each comes as a class of operations and a separate class of equations. The binders of a definition can thus list only the operation classes it needs to run, and the binders of a theorem additionally the equation classes it needs to be reasoned about. An effect of this is longer binder lists on the theorems. One can see this in the theorems quoted in Sections 4 to 8, for instance in `quicksort_correct` of Section 6, which carries five bracketed hypotheses.

Instance priorities. A hypothesis in square brackets, written `[C M]` for a class `C` and a monad `M`, asks Lean to find an instance of `C` for `M` by search rather than to receive it as an argument. As a toy example, suppose a class `Size` α provides one operation `size` : $\alpha \to \mathbb{N}$, and two instances are declared: a default one, for every type, with `size x = 0`, and one for lists, with `size L = L.length`. Now apply a function with the hypothesis `[Size` α`]` at $\alpha = \texttt{List}\,\beta$. Both instances fit: the default one, since it is declared for every type, so in particular for lists; and the list one directly. For $b : \beta$, `size [b]` could then be 0 or 1. Search collisions are thus possible and need disambiguation. To this end, Lean uses priorities: each instance carries a natural number, called its priority, say 100 for the default instance and 1000 for the list one, and among the fitting instances the search takes the one of highest priority. With these numbers, `size [b]` is 1, and a type with no instance of its own, `Bool` say, still has the default. We use this mechanism in Section 4 to give to a single program text two different behaviours for its tick calls. The first is a default instance, declared for every monad at low priority: under it, a tick call does nothing, being defined as `pure ()` (explained in Section 4), so the run behaves as if the tick were not there. The second is an instance at high priority for the monad transformer `TimeMT`, the cost-counting transformer named in the introduction and defined in Section 4: under it, the charged cost is added to the running total.

The extended nonnegative reals. Probabilities and expectations in this thesis take values in the extended nonnegative reals $[0, \infty]$, Mathlib's `ENNReal`. The type is chosen for one property: every sum of nonnegative terms converges in it, that is, for any index type I and family $x : I \to [0, \infty]$ the sum $\sum'_i x_i$ (Mathlib's `tsum`) is defined, possibly equal to ∞. As a consequence, the expectations computed in later sections, each a sum of the form $\sum'_a p(a)\, g(a)$ for a distribution p and a function g into $[0, \infty]$, with the expected cost of an algorithm as the main case, carry no summability side condition. The type inherits the arithmetic of $[0, \infty)$, where subtraction is already truncated at zero, and extends it to ∞ with the usual conventions, among them $0 \cdot \infty = 0$ and $a - \infty = 0$. The statements of our theorems inherit these conventions. The map `toReal` descends from $[0, \infty]$ to $\mathbb{R}$ and sends ∞ to 0, so restating a theorem in $\mathbb{R}$ through `toReal` is faithful only once the quantity involved is known to be finite. The cost theorems of later sections that descend to $\mathbb{R}$ therefore need to bound the quantity involved, typically the expected cost, to show that it is not ∞.

`PMF` and the discrete Giry monad. A distribution over a type α is a value of type `PMF` α, that is, a function $P : \alpha \to [0, \infty]$ with $\sum'_a P(a) = 1$ (Mathlib). What makes `PMF` usable as a programming type is that it is a monad, the one Section 2 announced, known as the discrete Giry monad. Its two operations `pure` and `bind` are defined as follows. For $v : \alpha$, `pure` v (which is of type `PMF` α) is the point mass at v, the distribution assigning probability 1 to v and 0 elsewhere. For a distribution P : `PMF` α and, for a type β, a function $f : \alpha \to$ `PMF` β, we have that `bind` averages: the probability of $b : \beta$ under `P >>= f` (which is of type `PMF` β) is

$$\sum'_{e:\alpha} P(e) \cdot (f\, e)(b),$$

the average, under P, of the probabilities $(f\,e)(b)$. The sum is a `tsum`, defined even when α is infinite, and this equation is Mathlib's lemma `PMF.bind_apply`. In words, `P >>= f` is the distribution of the following two-step experiment: draw from P and obtain e, then draw from $f\,e$. This formula is all that is needed to read a `do` block over `PMF`. To recall, a `do` block is a finite sequence of lines and a last line, written in the context of a monad. The kinds of lines and their desugaring are the same for every monad; what changes with the monad is what a line means. Here we take the time to give to each kind its meaning at `PMF`, because reading the monadic text of our algorithms for the analysis rests on it (the `IO` reading is already assumed understood). Throughout, `P` and `Q` stand for distributions (over possibly different types, left unstated: they depend on the block at hand); `t`, `u` and `v` for values of any type; `b` for a Boolean; `S`, `T` and $\mathtt{S}_0$, ..., $\mathtt{S}_n$ for `do` blocks; and `e`, `x`, `r` for names, the identifiers that lines declare and which become the variables of the `fun` binders after desugaring. With these declared, a `do` block starts with `do`, and its lines, after desugaring, fall into three categories. A line that is not the last becomes a `bind`, or a `let`. By the latter we mean the usual `let` expression of *Functional Programming in Lean*: `let x := t; body` is one term, whose value is the value of `body` with `x` standing for `t`, and it involves no `bind` and no `pure`. The last line is a term of the monad's type, here a distribution, and the `bind`s and `let`s of the lines above stand in front of it, so it is the innermost term when we desugar the block. For a `do` block to be well defined, two facts must hold, and they organise everything below. First, a `do` block is a term of the same type as its last line. Second, at each line, the lines below form again a block, hence again a distribution, which may use the values named above that line. Indeed, a `bind` in front of the rest of a `do` block has the type of that rest, and so does a `let`. Moreover, as a bind desugaring `P >>= fun e =>` or `P >>= fun _ =>` must typecheck, the body after the `=>` must be a distribution, and that body is the rest of the block, desugared. So the two facts hold by induction, from the last line up to the first. With this understood, we list below the kinds of lines:

- A bind, `let e` $\leftarrow$ `P`, draws a value from the distribution `P` and names it `e`. It desugars to `P >>= fun e =>`, followed by the rest of the block.
- An action is a bind whose value gets no name. The line is just a distribution `Q`; the block draws from it and nothing below uses the value. It desugars to `Q >>= fun _ =>`.
- A computation, `let x := t`, names the value `t` and draws nothing. It desugars to a `let`, that is, the line becomes `let x := t;` followed by the rest of the block.
- A case split chooses one block among finitely many: an `if b then S else T` between the two blocks `S` and `T`, or a `match t with | A₀ => S₀ | ... | Aₙ => Sₙ` among the blocks $\mathtt{S}_0$, ..., $\mathtt{S}_n$. Writing `S'`, `T'` and $\mathtt{S}'_0$, ..., $\mathtt{S}'_n$ for the desugared blocks, the line desugars to `(if b then S' else T') >>= fun r =>` or `(match t with | A₀ => S'₀ | ... | Aₙ => S'ₙ) >>= fun r =>`. The term in the parentheses is one distribution, the one of the block chosen, and the bind draws from it and names the drawn value `r` for the rest of the block.
- A `for` loop, for example over a list with two elements `y` and `z`, `for x in [y, z] do S`, runs the block `S` once per element. Each `S` may use the name `x`, which stands for `y` in the first run and for `z` in the second. It desugars to one bind per element: writing `S' y` (resp. `S' z`) for the desugared body with `y` (resp. `z`) substituted for `x`, the loop becomes `S' y >>= fun _ => S' z >>= fun _ =>`. Each of `S' y` and `S' z` is itself a block, and Lean requires its last line to have a value of type `Unit`, that is, `()`. By the first fact, each is then a distribution over the one-element type `Unit`. The drawn value can only be `()`, and no line below uses it, so each bind discards it with the underscore, as in an action line.
- A mutable variable, `let mut x := t`, may be reassigned in a later line `x := u`. Both lines desugar to `let`s, as before, so below the reassignment `x` stands for `u` and above it for `t`. One case is special: a reassignment inside a case split or a `for` loop, where the branch taken decides the new value. All the desugaring does there is pass that new value over to the lines after the split, so that the reassignment is not forgotten; the `let` alone could not do it because it reaches only to the end of its block. Lean therefore ends each block of the split with `pure` of the latest value of `x`, and the split's bind draws exactly that value, with its binder named `x` rather than `r` or `_`.

- The last line is `return v`, which is just `pure v`, the point mass at v; it differs from the bind `let e ← pure v` only in that it marks the end of the block, so we do not need the name `e` for the lines below, since there are none.

Here is a desugaring example:

sugared	desugared
`do`	
`  let e ← P`	`P >>= fun e =>`
`  Q`	`Q >>= fun _ =>`
`  let x := t`	`let x := t;`
`  x := u`	`let x := u;`
`  if b then S else T`	`(if b then S' else T') >>= fun r =>`
`  match t with | A₀ => S₀ | ...`	`(match t with | A₀ => S₀' | ...) >>= fun r =>`
`  for x in [y, z] do S`	`S' y >>= fun _ => S' z >>= fun _ =>`
`  return v`	`pure v`

In a bind line `P >>= f`, we call f the *continuation* of the line, that is, f is the rest of the block as a function of the drawn value. With the `do` block explained at `PMF`, one has the useful interpretation of binding that Section 2 already announced. Namely, one must see the bind as an *update*: the distribution of a block is the distribution of its first line `P`, which updates its continuation `f`: `P >>= f`. The update is backward: the continuation at each drawn value e is itself a block, so its distribution $f\,e$ must be known first, and the distribution of the whole block is assembled from the last line upwards. This direction is the one our proofs use, because it fits recursion. `PMF` is moreover a lawful monad: Mathlib proves for it the three equations of `LawfulMonad` (the same as at the start of this section). The associativity equation lets the update be read in the other direction as well: one holds the distribution built by the lines read so far, and each bind updates it forward by one draw. Indeed the equation states that `(m >>= f) >>= g` equals `m >>= fun a => f a >>= g`. Also, the left identity equation removes a deterministic line: a draw from a point mass, `pure t >>= f`, is `f t`, so the line disappears into a substitution. Finally, Mathlib provides the uniform distribution on a finite nonempty type α, `PMF.uniformOfFintype`, which assigns probability $1/|\alpha|$ to each value of α. On the type $\alpha =$ `Fin n` (the integers $0, \ldots, n-1$) it is the distribution the framework assigns to its primitive draw (Section 4). An equality of distributions is a pointwise equality: two distributions are equal when they assign the same probability to every value, and when a later section states that an output distribution is a point mass, or equals a stated distribution, it is this pointwise equality that is proved.

Computable and noncomputable definitions. Lean processes a definition in two pipelines: the elaborator and the kernel, which check the definition and the proofs about it, and the compiler, which turns it into executable code. A definition may carry the keyword `noncomputable`; it then goes through the first pipeline only, and Lean generates no code for it. `PMF` is noncomputable. Indeed, its values live in $[0, \infty]$, a type built on the real numbers, and the definitions of the real numbers in Mathlib carry the keyword `noncomputable`, so every definition built on them inherits it, and Mathlib declares the whole of `PMF` noncomputable. Moreover, as seen, the `bind` of `PMF` is a `tsum`, an infinite sum, for which Lean generates no executable code either. Running an algorithm therefore needs a relevant monad other than `PMF`. A computable type of distributions, such as some probabilistic programming libraries use, would be one candidate. These are distributions with finite support: a distribution is then a finite list of pairs of a value and a nonnegative rational weight, and the operations `pure` and `bind` are defined as for `PMF`, with a finite sum in place of the `tsum`. A countably infinite support, such as that of the geometric distribution, needs infinite sums that no program computes exactly. We did not take that route: it loses the analysis Mathlib provides on $[0, \infty]$, and our distribution reading is meant to be proved about, not run, although such a type would have one attraction, computing on a given input the tree of the runs of the algorithm, each branch labelled by its probability. The relevant monad used for execution is the standard `IO` one. An algorithm whose monad is `IO` compiles and runs, in the editor through the command `#eval`, which Section 7 uses, or as a standalone executable through a `main` function. In it a random draw is a call

to the function `IO.rand`. This function keeps a pseudorandom generator in a mutable reference: at each call it reads the generator, computes a value from it, and writes the updated generator back. The reference is initialised when the program starts, with a seed of eight random bytes that Lean requests from the operating system. So a monad for execution exists already; but why would one want to run the algorithm at all, when the theorems are about its distribution and its cost, neither of which needs the compiler? This is the question the introduction deferred to this section. Three reasons made an executable reading seem worth keeping to us. The first is that it is not difficult to obtain: the reading is one type ascription, the text being already there. The second is that it is useful as it stands: a run on an example tests the definition and shows what the tick calls count on that instance. Such runs are a cheap check of the placement of the ticks and support the trust Section 5 asks for; so does comparing the average cost over many runs with a proved bound, usually a well-known textbook one, since agreement is further evidence that the ticks are well placed. The third is that an executable text lets an automated agent test and compute with the very text it reasons about; Section 9 returns to this idea in more detail.

4 The framework

We present the framework through the toy algorithm that Section 7 analyses in full. `RandMax` finds the maximum of a list: it draws a uniformly random position, charges one unit for a comparison, and takes the maximum of the element at that position and of the maximum of the rest. After this section, every notation involved here will be clear.

```
def RandMax {M} [Monad M] [RandMonad M] [MonadCost ℕ M] :
    List α → M α
  | [] => return default
  | L@(_ :: _) => do
      let idx ← randIdx L
      MonadCost.tick 1
      let m ← RandMax (L.eraseIdx idx)
      return max L[idx] m
  termination_by L => L.length
  decreasing_by all_goals grind
```

The type variable α carries a linear order and a default element for the empty list. The clause `termination_by` names the measure that shrinks: the length of the list. `decreasing_by` discharges the proof that it shrinks at every recursive call, with `grind` closing each goal. This is our single definition every theorem of Section 7 is about: the correctness theorem, the distribution of the cost, the exact expected cost and the tail bound all name this one term. Moreover, as you see, the design is entirely in the constraint that one can put on the monad `M`. Here already there are two typeclasses, `[RandMonad M]` and `[MonadCost ℕ M]`. Two more typeclasses will be introduced below and are used for the formal analysis.

The definitions below use the following objects. `M` is a monad, a type constructor `M : Type → Type` with `pure` and `bind`. α and β are arbitrary types; unlike for `RandMax`, no linear order is assumed on them. $n \geq 1$ is a natural number, and `Fin n` is the type of the integers $0, \ldots, n-1$. `C` and `T` are generic types which are interpreted as the type of costs (or time, if you prefer).

Definition 4.1 (Random monad)**.** The first typeclass that we define is `[RandMonad M]`. A random monad is a monad `M` with a "source of randomness": one primitive "draw" that is in reality just an element of `M (Fin n)`, for every $n \geq 1$. From it we derive `randIdx` ("drawing" an index of a nonempty list).

In Lean:

Class RandMonad and the sampler randIdx

```
/-- A random monad: a monad ‘M‘ with one function ‘randFin‘, its source
of randomness. -/
class RandMonad (M : Type → Type) [Monad M] where
  /-- An element of ‘M (Fin n)‘. -/
  randFin (n : ℕ) [NeZero n] : M (Fin n)
```

```
/-- ‘randFin‘ at the length of a nonempty list. -/
def randIdx {M} [Monad M] [RandMonad M] {α}
    (L : List α) (h : 0 < L.length := by grind) : M (Fin L.length) :=
  have : NeZero L.length := ⟨h.ne'⟩
  RandMonad.randFin L.length
```

For now, `randFin` is just a function, and there is no concept of drawing or uniformity. This arrives later, as an axiom of `LawfulRandMonad`, and keeping it out here is what lets `IO` implement the class at all. We give the following instances of `RandMonad`: at `IO` the draw reads the system generator through `IO.rand`; at `PMF` it is the uniform distribution on `Fin n`:

The RandMonad instances of IO and PMF

```
/-- ‘IO‘ has real computable (pseudo-)randomness. -/
instance : RandMonad IO where
  randFin n := do
    let i ← IO.rand 0 (n - 1)
    return ⟨i % n, Nat.mod_lt i (Nat.pos_of_ne_zero (NeZero.ne n))⟩

noncomputable instance : RandMonad PMF where
  randFin n :=
    have : Nonempty (Fin n) := ⟨⟨0, Nat.pos_of_ne_zero (NeZero.ne n)⟩⟩
    PMF.uniformOfFintype (Fin n)
```

Remark 4.2. Each time we use `randIdx` on a nonempty list `L`, we need to provide a proof that `0 < L.length`. This is thankfully discharged by `grind` by default.

In our case studies, every derived “draw” is built from `randFin`. We have built on it the uniform index into a list, the random bit and the vector of bits, and the uniform element of a finite set through its enumeration. You may ask: why did we choose `Fin n` as the type of the “draw”, and not a `Finset` of n elements? Because the `IO` reading could not implement such a primitive: there would be no instance `RandMonad IO`. Indeed a `Finset` is a list quotiented by permutation, and the rule of quotients is that a function out of a quotient of A by a relation r (that is, a function whose domain is that quotient) is exactly a function on A invariant under r (i.e. sending r-related elements to the same value). Here r relates the lists that are permutations of each other, so a function out of a `Finset` must be invariant under permutation. However, a program that draws from a list is not invariant: at `IO`, drawing from `[a, b]` and drawing from `[b, a]` are obviously different programs (on the draw of index 0 one returns a and the other b), even though the output distribution of the draw is the same. Classical choice would pick an element regardless, but it does not compute, and the `IO` reading must. Still, a uniform draw from a `Finset` does exist as a distribution, Mathlib's `PMF.uniformOfFinset`.

Remark 4.3. The constraint of not being able to draw from a `Finset` is one of the two reasons why, in Section 8, our multigraph definition has its edges as a list (to be able to draw them in Karger's algorithm).

So far nothing assigns a probability to anything. That is the job of the following proof-side class.

Definition 4.4 (Lawful random monad)**.** A lawful random monad is a lawful monad `M` which is a random monad (i.e. `[LawfulMonad M]` and `[RandMonad M]`), together with a monad morphism `toPMF` : $M\,\alpha \to$ `PMF` α (i.e. it sends the `pure` and the `bind` of `M` to those of `PMF`) and we impose the condition that it sends `randFin n` to the uniform distribution on `Fin n`.

In Lean:

Class LawfulRandMonad

```
class LawfulRandMonad
    (M : Type → Type) [Monad M] [LawfulMonad M]
    extends RandMonad M where
  /-- Interpret an ‘M‘-computation as a probability distribution. -/
  toPMF : ∀ {α}, M α → PMF α
  /-- ‘pure a‘ maps to the Dirac mass at ‘a‘. -/
  toPMF_pure : ∀ {α} (a : α), toPMF (pure a) = pure a
  /-- ‘bind‘ distributes through ‘toPMF‘. -/
  toPMF_bind : ∀ {α β} (x : M α) (f : α → M β),
    toPMF (x >>= f) =
```

```
      (toPMF x) >>= (fun a => toPMF (f a))
  /-- `randFin n` maps to the uniform distribution on `Fin n`
  (`Nonempty (Fin n)` is synthesized from `[NeZero n]`). -/
  toPMF_randFin :
    ∀ (n : ℕ) [NeZero n],
      toPMF (randFin n) = PMF.uniformOfFintype (Fin n)
```

The last equation is the axiom of uniformity. As in the case of `RandMonad`, `toPMF` is a field of the class, so every instance must supply such a morphism. We give the instance for `PMF` here: `toPMF` is the identity, and the three equations hold by `rfl`.

In Lean:

The LawfulRandMonad instance of PMF

```
noncomputable instance : LawfulRandMonad PMF where
  toPMF := id
  toPMF_pure _ := rfl
  toPMF_bind _ _ := rfl
  toPMF_randFin _ := rfl
```

As we said earlier, the monad `IO` fails to instantiate this class, because it fails at `toPMF` and at the uniformity equation. `IO` is a lawful monad, but an `IO` term may read a file, fail or depend on the state of the machine. A term that only fails can be given the type `IO Empty`, and no distribution on the empty type exists, since a sum over no values is 0, not 1 (so there is no `PMF Empty` and hence no `toPMF : IO Empty → PMF Empty`). The last equation also does not hold: `IO.rand` reads a mutable generator and draws through Lean's `randNat`, which is only approximately uniform.

Now, it helps to interpret a term $m : M\,\alpha$ as a program, and `toPMF m` as the distribution of its output. The `LawfulRandMonad` class is used everywhere in the analysis: we state every theorem of the framework over an arbitrary `LawfulRandMonad M`. From it we define three notations. Let `e` be an algorithm read at `M`; then $\mathcal{D}_M[e]$ is its output distribution, and $\mathbb{P}_M[e = v]$ and $\mathbb{P}_M[e \in S]$ are the probabilities that its output equals `v` or lies in the event S. Here `prob p S`, the probability of the event S under the distribution `p`, is the sum of the probabilities of the elements of S.

In Lean:

prob and the three notation macros

```
/-- Probability of the event `s` under the distribution `p`. -/
noncomputable def prob {α : Type*} (p : PMF α) (s : Set α) : ENNReal :=
  ∑' a, s.indicator p a

scoped macro "𝒟_{" M:term "}[" e:term "]" : term =>
  `(LawfulRandMonad.toPMF ($e : $M _))

scoped macro "ℙ_{" M:term "}[" e:term:51 " = " v:term:51 "]" : term =>
  `(LawfulRandMonad.toPMF ($e : $M _) $v)

scoped macro "ℙ_{" M:term "}[" e:term:51 " ∈ " S:term:51 "]" : term =>
  `(prob (LawfulRandMonad.toPMF ($e : $M _)) $S)
```

All three are macros and not abbreviations. This is important because this means they expand automatically to the underlying `toPMF` and `prob` terms, so there is no need for unfolding during a proof. Each of the three has a twin without the monad index, $\mathcal{D}[m]$, $\mathbb{P}[m = v]$ and $\mathbb{P}[m \in S]$, for a computation m whose type already names the monad, where the ascription would be redundant. Sections 6 and 7 use both forms.

Definition 4.5 (Cost interface)**.** A cost monad is a monad `M` with one operation `tick`, of type $C \to M\,\texttt{Unit}$, where `C` is the type of the costs. The tick call "`tick c`" charges the cost $c : C$ where it stands. What such a call does is left to the instance: the default instance discards it.

In Lean:

Class MonadCost and its default instance

```
/-- A typeclass for monads with a cost-charging operation. -/
class MonadCost (C : Type) (M : Type → Type) where
  /-- Charge a cost of `c`. -/
  tick : C → M Unit
```

```
/-- Default instance: ticking is a no-op (‘pure ()‘).
This is used when we don’t want to track the cost.
Low priority so that the ‘TimeMT‘ instance takes precedence. -/
instance (priority := 100) instMonadCostDefault
    {C} {M} [Monad M] : MonadCost C M where
  tick _ := pure ()
```

Remark 4.6. Why did we make an instance whose tick is invisible? Simply because we wanted to write the algorithm once: the tick then stands in the text in any case, and running that text without carrying a cost needs an instance that discards the tick.

The class is generic in the cost type `C`. However, in every one of our analyses we fixed $C = \mathbb{N}$.

Definition 4.7 (Lawful cost model)**.** A cost model on a lawful random monad is lawful when its ticks are invisible to the output distribution: for every cost $c : C$, `toPMF (tick c)` is the point mass at `()`. The default instance satisfies this.

In Lean:

Class LawfulMonadCost

```
class LawfulMonadCost (C : Type) (M : Type → Type)
    [Monad M] [LawfulMonad M] [LawfulRandMonad M] [MonadCost C M] :
    Prop where
  /-- ‘tick‘ maps to the Dirac mass at ‘()‘. -/
  toPMF_tick (c : C) :
    LawfulRandMonad.toPMF (MonadCost.tick c : M Unit) = PMF.pure ()
```

Definition 4.8 (Timed monad transformer)**.** `TimeMT` is built in two stages. To understand it, we first need the structure `TimeM T` α from CSLib, which is a wrapper for a value `ret` : α and a cost `time` : T. For a fixed `T` with a zero and an addition, `TimeM T` is a monad: its `pure` has cost 0 and its `bind` adds the costs of its two stages.

In Lean:

Structure TimeM from CSLib, with pure and bind

```
@[ext]
structure TimeM (T : Type*) (α : Type*) where
  /-- The return value of the computation -/
  ret : α
  /-- The accumulated time cost of the computation -/
  time : T

protected def pure [Zero T] {α} (a : α) : TimeM T α :=
  ⟨a, 0⟩

protected def bind {α β} [Add T] (m : TimeM T α) (f : α → TimeM T β) : TimeM T β :=
  let r := f m.ret
  ⟨r.ret, m.time + r.time⟩
```

What we would like from `TimeMT` is to implement a tick instance for it that makes `TimeMT` track the cost of a run in `M` next to its result. So, we want that, when an algorithm runs in `M`, each tick of an operation adds its cost to a running total, and that at the end we read both the result and the total. A naive way to get this is to have a program in `M` whose output value is a `TimeM T` α, the wrapped pair `(a, t)`. That is, we want a term of the type `M (TimeM T` α`)`. Now, since we want to track these costs with a well-defined tick, we need to first put `M (TimeM T` α`)` inside a structure, which is a new type, and then secondly define the `pure` and the `bind` of that type ourselves. Only then will we define its tick.

So the first stage is the wrapper. A `TimeMT T M` α is a structure with one field, “a run” (or “a computation”), `run` : M (`TimeM` T α).

In Lean:

Structure TimeMT

```
/-- Monad transformer that threads a time cost ‘T‘ through an
    arbitrary base monad ‘M‘. -/
@[ext]
structure TimeMT (T : Type u) (M : Type (max u v) → Type w) (α : Type v) : Type w where
```

```
  run : M (TimeM T α)
```

The second stage is the operations: the `pure` and the `bind` we define ourselves.

In Lean:

The operations of TimeMT: pure and bind

```
namespace TimeMT

variable {T : Type} {M : Type → Type} {α β : Type}

protected def pure [Zero T] [Pure M] (a : α) : TimeMT T M α :=
  ⟨Pure.pure ⟨a, 0⟩⟩

protected def bind [Add T] [Monad M]
    (m : TimeMT T M α) (f : α → TimeMT T M β) : TimeMT T M β := ⟨do
    let ⟨a, t1⟩ ← m.run
    let ⟨b, t2⟩ ← (f a).run
    pure ⟨b, t1 + t2⟩⟩
```

Here is a description of these operations: `pure a` returns the pair `(a, 0)` with the `pure` of `M` while `bind` runs `m.run` in `M`, which gives the value `a` and the cost `t1` of `m`, feeds `a` to `f`, runs `(f a).run` in `M`, which gives the value `b` and the cost `t2` of `f a`, and returns the pair `(b, t1 + t2)`, again with the `pure` of `M`. So now we can track the cost: a `do` block over `TimeMT` is a chain of these binds, and the costs add up along it. And this does not mix with the `pure` and the `bind` of the base monad `M`, which only run the two computations `m` and `f a`. `TimeMT T M` is a monad as soon as `T` has a zero and an addition, and a lawful monad when `T` is an additive monoid and `M` is lawful.

Two more functions complete it. The first is `lift`: it takes a computation `ma` : $M\,\alpha$ of `M` and returns the element of `TimeMT T M` α whose field `run` is `ma` with its returned value paired with the cost 0 (the `<$>` in the code), so that an algorithm read at `TimeMT T M` can still use the operations of `M`, by executing an operation in `M` and then lifting it. It helps to think of `lift ma` as `ma` run at cost 0: for instance the draw `randFin` of `M`, once lifted, is a draw of `TimeMT T M` that costs nothing. The second is the wanted `tick`: `tick t` is the element of `TimeMT T M Unit` whose field `run` is `pure` $\langle(), t\rangle$, the `pure` of `M` at the pair of the value `()` and the cost `t`. It helps to think of it as the computation that does nothing and costs `t`.

In Lean:

lift and tick

```
def lift [Zero T] [Functor M] (ma : M α) : TimeMT T M α :=
  ⟨(fun a => ((⟨a, 0⟩ : TimeM T α)) <$> ma)⟩

def tick [Monad M] (t : T) : TimeMT T M Unit :=
  ⟨Pure.pure ⟨(), t⟩⟩
```

The last missing piece is the `MonadCost` instance of `TimeMT`: its `tick` is `TimeMT.tick`, and its priority, 1000 against 100, is what selects it over the default instance, as discussed in Section 3.

In Lean:

The MonadCost instance of TimeMT

```
/-- ‘TimeMT‘ instance: ticking accumulates cost via ‘TimeMT.tick‘,
for any cost type ‘T‘. Higher priority than the default no-op
instance. -/
instance (priority := 1000) instMonadCostTimeMT
    {T : Type} {M} [Monad M] : MonadCost T (TimeMT T M) where
  tick := TimeMT.tick
```

Furthermore, we have that `TimeMT ℕ M` is a random monad when `M` is one, with the lifted draw of `M` being its source of randomness. It is also a lawful random monad when `M` is one, with its `toPMF` being the map that forgets the time field and then applies the `toPMF` of `M`.

In Lean:

The RandMonad and LawfulRandMonad instances of TimeMT ℕ

```
instance instRandMonadTimeMT {M} [Monad M] [RandMonad M] :
```

```
    RandMonad (TimeMT ℕ M) where
  randFin n := TimeMT.lift (RandMonad.randFin n)

noncomputable instance instLawfulRandMonadTimeMT
    {M} [Monad M] [LawfulMonad M] [inst : LawfulRandMonad M] :
    LawfulRandMonad (TimeMT ℕ M) where
  toRandMonad := instRandMonadTimeMT
  toPMF m := inst.toPMF (TimeM.ret <$> m.run)
  toPMF_pure a := by rw [TimeMT_erase_pure, inst.toPMF_pure]
  toPMF_bind m f := by rw [TimeMT_erase_bind, inst.toPMF_bind]
  toPMF_randFin n := by
    intro hn
    rw [randFin_timeMT, TimeMT_erase_lift, inst.toPMF_randFin]
```

The three equations are proved with some lemmas that we made about `TimeMT` (`TimeMT_erase_pure`, `TimeMT_erase_bind` and `TimeMT_erase_lift`), and `randFin_timeMT` is simply the equation, true by `rfl`, that the draw of `TimeMT ℕ M` is the lifted draw of `M`, as the instance above defines it.

The `TimeMT` instance of `MonadCost` is a lawful cost model as long as `M` is a lawful random monad: the `toPMF` of `TimeMT ℕ M` forgets the time field of `pure` $\langle$`(), c`$\rangle$, the `run` of `tick c`, which leaves `pure ()`, and `toPMF (pure ())` is `PMF.pure ()` (since `toPMF` is a monad morphism).

We summarise for the four monads which of them implements which class:

monad	`LawfulMonad`	`RandMonad`	`LawfulRandMonad`	`MonadCost`	`LawfulMonadCost`
`IO`	yes	yes	no	default	no
`PMF`	yes	yes	yes	default	yes
`TimeMT ℕ IO`	yes	yes	no	`TimeMT`	no
`TimeMT ℕ PMF`	yes	yes	yes	`TimeMT`	yes

and let us furthermore summarise what each class operation becomes at each monad. That is what the four "readings" are:

reading	`randFin`	`tick`	`RandMax L` is
`IO`	the system generator	discarded	an executable program
`PMF`	the uniform distribution	discarded	the distribution of the output (here the point mass at the maximum of `L`)
`TimeMT ℕ IO`	lifted from `IO`	accumulated	an executable program paired with a cost
`TimeMT ℕ PMF`	lifted from `PMF`	accumulated	the joint distribution of output and cost

Definition 4.9 (Joint distribution and its marginals)**.** Let us fix an m : `TimeMT` $\mathbb{N}\, M\, \alpha$ (a "timed program"). Over a lawful random monad `M`, its distribution p over the pairs r : `TimeM` $\mathbb{N}\, \alpha$ is `toPMF (m.run)`, which we name `TimedPMF m`. We define `costPMF m` to be its time-marginal: the image of p under the projection $r \mapsto \mathrm{time}(r)$. Its other marginal is similarly the image of p under $r \mapsto \mathrm{ret}(r)$. We define the expected cost of m to be $\mathbb{E}_M[\mathrm{cost}\ m] = \sum'_r p(r) \cdot \mathrm{time}(r)$ in $[0, \infty]$, the sum ranging over all pairs r. This is simply the expectation of the time-marginal.

In Lean:

TimedPMF, expected_cost and costPMF

```
noncomputable def expected_cost {α : Type} (p : PMF (TimeM ℕ α)) : ENNReal :=
  ∑' (res : TimeM ℕ α), p res * (res.time : ENNReal)

noncomputable abbrev TimedPMF
    {M : Type → Type} [Monad M] [LawfulMonad M]
    [inst : LawfulRandMonad M] {α : Type}
    (m : TimeMT ℕ M α) : PMF (TimeM ℕ α) :=
  inst.toPMF m.run

scoped macro "𝔼_{" M:term "}[cost " e:term "]" : term =>
  `(expected_cost (TimedPMF ($e : TimeMT ℕ $M _)))

noncomputable def costPMF {M : Type → Type} [Monad M] [LawfulMonad M]
    [LawfulRandMonad M] {α : Type} (m : TimeMT ℕ M α) : PMF ℕ :=
  (TimedPMF m).map TimeM.time
```

Like the output notations, $\mathbb{E}_M[\mathrm{cost}\ e]$ has a twin without the monad index, $\mathbb{E}[\mathrm{cost}\ m]$, for a computation m already typed at `TimeMT ℕ M`. Sections 6 and 7 use both forms.

Remark 4.10. The output marginal of p has no definition of its own in the code because we

already have access to it. It is the `toPMF` of the instance `LawfulRandMonad (TimeMT ℕ M)`, that is, `toPMF (TimeM.ret <$> m.run)` (the image of `TimedPMF m` under $r \mapsto \mathrm{ret}(r)$). It is also equal to $\mathcal{D}_{\texttt{TimeMT}\,\mathbb{N}\,M}[m]$. Every lemma about `toPMF` thus applies to it. The time-marginal needs a name because no class can produce it. Indeed, the type of `toPMF` (i.e. $M\,\alpha \to \texttt{PMF}\,\alpha$) shows that `toPMF` only ever gives a distribution over the output type (and not the cost one), whatever the instance.

Remark 4.11. Let `e` be an algorithm (that is, a text polymorphic in its monad), and `M` a lawful random monad. Then $\mathcal{D}_M[e]$ reads `e` at `M` with the default instance of `MonadCost`, whose tick is `pure ()`, and $\mathcal{D}_{\texttt{TimeMT}\,\mathbb{N}\,M}[e]$ reads it at `TimeMT ℕ M`, where the ticks accumulate, and then forgets the time field. Both are the notation $\mathcal{D}_N[e]$ at a lawful random monad `N` with a lawful cost model, for $N = M$ and $N = \texttt{TimeMT}\,\mathbb{N}\,M$. Therefore, any theorem about $\mathcal{D}_N[e]$ stated for every such `N` (as our correctness theorems are) says the same thing of both distributions. For instance `quicksort_correct` at $N = \texttt{TimeMT}\,\mathbb{N}\,\texttt{PMF}$ is the theorem `quicksort_correct_timed_pmf` of the repository. You may have wanted to state the equation $\mathcal{D}_{\texttt{TimeMT}\,\mathbb{N}\,M}[e] = \mathcal{D}_M[e]$. This equation holds true since the two readings share the text of `e`, and the `toPMF` of `TimeMT ℕ M` forgets exactly what the ticks add. But we cannot state it for every `e` at once since it is a fact about polymorphic definitions that Lean does not prove internally. So we cannot make such a general bridge for any `e`. Although for one given algorithm `e` it is possible to prove it, it needs to be proved (possibly by induction) on that algorithm. We decided not to do so, and this is fine, since we never needed it: as said above, a correctness theorem stated over a lawful random monad with a lawful cost model already covers both readings.

This is all there is to know about the framework itself. The objects of this section are its design, and in the repository they live in six files of `ARA/Infrastructure/`. In `Randomness/` these are `LawfulRandMonad.lean` and `Prob.lean`, for the random monads and the output notations. In `Complexity/`, for the cost side, these are `MonadCost.lean`, `TimeMT.lean`, `TimedSemantics.lean` and `ExpectedCost.lean`, together with `TimeM` from CSLib. The nine other files of `Infrastructure/` are building on these objects without changing them. They hold the lemmas about them and the automation built on them (the tactics that Section 6 names). A few more definitions are also derived from them: some samplers built from `randFin` (that we talked about at the beginning of this section), the amplification combinator of an algorithm and the tail bounds of Section 6, and the geometric distribution of Section 8.

5 The trust boundary

A reader of an ARA cost theorem must trust two things. The first is the Lean kernel, the standard assumption of any Lean development. The second is specific to the framework: that each tick call correctly reflects the cost of the algorithm step next to it, in its two parts, how much it charges and where it stands. Indeed, nothing checks a tick against the work the surrounding code does. It is not tied to the kernel or to any true computation semantics. As you have seen in the previous section, the plumbing we designed for the ticks is essentially for the syntactic declaration of the ticks to accumulate correctly. Clearly, there can be a discrepancy between what the operations really are and what the tick charges. For example, take our Quicksort algorithm:

The program text of Quicksort

```
def Quicksort
    {M} [Monad M] [RandMonad M] [MonadCost ℕ M] :
    List α → M (List α)
  | [] => return []
  | L@(_::_) => do
      let idx ← randIdx L
      let pivot := L[idx]
      let rest := L.eraseIdx idx
      let L1 := rest.filter (· < pivot)
      let L2 := rest.filter (· ≥ pivot)
      MonadCost.tick rest.length
      let S1 ← Quicksort L1
      let S2 ← Quicksort L2
      return (S1 ++ [pivot] ++ S2)
  termination_by L => L.length
```

```
  decreasing_by all_goals grind
```

The text has one tick call. It charges `rest.length` units, one per comparison of a remaining element with the pivot. But the two lines above it compute the partition with two `filter` traversals, so the text performs twice the comparisons the tick declares. This is a discrepancy, but it is fine once we read the code and trust that one traversal would do: a single pass over `rest`, comparing each element with the pivot and putting it on the right side (Lean's `List.partition`), defines the two sublists `L1` and `L2` with exactly `rest.length` comparisons. Two more algorithms that we formalised show similar discrepancies: Quickselect, and Freivalds' algorithm [Fre77]; the latter tests whether $AB = C$ for three $n \times n$ matrices by drawing a random 0/1 vector r and checking $A(Br) = Cr$.

Every cost theorem in this thesis is therefore true of the cost model as declared; whether that model matches the code is for the reader to check, not for the kernel. The work thus divides in two. The kernel checks the proofs and the reader checks the ticks. This second check is small: it is linear in the length of the algorithm. Also, it can be supported through runs, and through repeated runs whose average cost one can compare against a textbook bound. This support is the second of the reasons for an executable reading given in Section 3. This trust problem concerns the cost theorems only. A correctness theorem is stated with the `LawfulMonadCost` hypothesis, so it holds wherever the ticks are and whatever they charge. Removing the trust assumption altogether is possible in principle, but only once the meaning of the program is itself formalised in Lean, so that the kernel can check the ticks; Section 9 names the work in that direction.

6 Correctness and cost statements

Every theorem the framework states is about one of the two marginals of Section 4: the distribution of the output or the distribution of the cost. Working through the case studies we noticed that the correctness statements kept returning in three tiers, and we built the correctness API around that observation. The classification, which is not new, follows the classical Las Vegas and Monte Carlo distinction [MR95]. It seems enough to let a reader place any correctness theorem of Section 8 within this classification. The theorems below are quoted from the repository with their proofs omitted. All algorithms below come from our case studies in Section 8. We assume that the reader knows these algorithms, although we will describe briefly what they do.

6.1 The three correctness tiers

The first tier is the Dirac tier, that of the Las Vegas algorithms. The algorithm's output distribution is the point mass at an element which has some specification. Quicksort satisfies this and works as follows: it draws a uniformly random pivot, partitions the remaining elements around it and recurses on both sides and concatenates the results in the right order. For Quicksort, the specification of the element with the concentrated mass is `sortSpec L`, the sorted permutation of the input list. This is the object of the following statement.

Theorem 6.1 (Quicksort sorts on every run)**.** For every lawful random monad M and every list L, the output distribution of `Quicksort L` is the Dirac distribution at `sortSpec L`.

In Lean:

sortSpec

```
lemma exists_sortedLE_perm (L : List α) :
    ∃ S : List α, S.SortedLE ∧ S.Perm L
  := by ...

noncomputable def sortSpec (L : List α) : List α :=
  (exists_sortedLE_perm L).choose
```

quicksort_correct

```
theorem quicksort_correct
    {M} [Monad M] [LawfulMonad M] [LawfulRandMonad M]
```

```
    [MonadCost ℕ M] [LawfulMonadCost ℕ M] (L : List α) :
    𝒟_{M}[Quicksort L] = PMF.pure (sortSpec L)
  := by ...
```

`SortedLE` and `Perm` are Mathlib's predicates for being sorted and for being a permutation of a list, and `choose` picks a witness of the existence lemma. Here a specification is a value of the output type (because it needs to be of the form `pure v`); for Quicksort, this is a list. It may be given by a definite description, as `sortSpec L` is: a list that is sorted and a permutation of L, picked by `choose` from the existence lemma.

Remark 6.2. Notice that the sorted permutation of a list is unique, so we may ask why we need `choose`: but even with uniqueness, `choose` is needed. Indeed a proof of "exactly one list is sorted and a permutation of L" is a proof, not a list.

The second tier is the exact-distribution tier, which deserves a name of its own: the output distribution of the algorithm is not a Dirac one, and that distribution is itself the correctness theorem. Reservoir sampling satisfies this and works as follows: it makes one pass through a list whose length it need not know in advance, holding one element, and replaces the held element by the i-th one with probability $1/i$.

Theorem 6.3 (Reservoir sampling is uniform)**.** For every lawful random monad M, every list L and every element a, a run of `reservoir` on L returns a with probability $\mathrm{count}(a)/|L|$. In particular, on a list with distinct entries every element is returned with probability exactly $1/|L|$.

In Lean:

reservoir_correct

```
theorem reservoir_correct
    {M} [Monad M] [LawfulMonad M] [inst : LawfulRandMonad M]
    [MonadCost ℕ M] [LawfulMonadCost ℕ M] [DecidableEq α]
    (L : List α) (a : α) :
    ℙ_{M}[reservoir L = some a] =
      (L.count a : ENNReal) /
        (L.length : ENNReal)
  := by ...
```

The Lean statement is heavier than the English one: a probability is a value of a `PMF`, so it lives in $[0, \infty]$, and the two natural integers have to be cast into it. Section 9 records a direction that would lighten such statements.

The third tier is the support-plus-probability tier, that of the Monte Carlo algorithms, which are "allowed to be wrong". It consists of two theorems: a "support claim" and a bound on the probability of being right, equivalently on the probability of erring. The first theorem, the support claim, is a statement about the support of the output distribution (the set of values returned with nonzero probability). In our case studies it has the following form: every output with nonzero probability satisfies a property that rules out one kind of error. Karger is such an algorithm, and there the property is that the reported value is never below the minimum cut value, so a wrong answer is always too large: the error is confined to one side of the order on the reported values. This is an example of what is called a "one-sided error" in the literature. Karger's algorithm repeatedly draws a uniformly random edge of a multigraph and contracts it, merging its two endpoints into one vertex, until two vertices remain or no edge does; it reports the partition of the original vertices that have merged within the surviving vertices, and the number of surviving edges. The surviving vertices can be more than two: the multigraph need not be connected, and the loop then stops because the edges ran out. Its theorems assume three things: that the graph is well-formed, that is, every endpoint of an edge is a vertex and no edge is a loop; that it has at least two vertices; and that the pick function, which chooses the vertex the contracted edge merges into, never chooses one of the untouched vertices (Section 8 explains this unusual pick function and the need for this hypothesis).

Theorem 6.4 (Karger's support claim)**.** Every output (P, v) of a run of Karger on a well-formed graph with at least two vertices is a partition P of the vertices into at least two blocks, each block is a cut of value exactly v, and v is at least the minimum cut value.

In Lean:

IsCutOutput and karger_isCut

```
structure IsCutOutput (g : MultiGraph α) (o : Finset (Finset α) × ℕ) : Prop where
  partition : g.IsCutPartition o.1
  sides : ∀ S ∈ o.1, g.IsCut S ∧ g.cutValue S = o.2

theorem karger_isCut
    {M} [Monad M] [LawfulMonad M] [inst : LawfulRandMonad M]
    [MonadCost ℕ M] [LawfulMonadCost ℕ M]
    {pick : MultiGraph α → Sym2 α → α}
    (hfresh : Fresh pick) (g : MultiGraph α) (hwf : g.WF) (h2 : 2 ≤ g.verts.card) :
    ∀ o ∈ 𝒟_{M}[Karger pick g].support,
      g.IsCutOutput o ∧ g.minCutValue ≤ o.2
  := by ...
```

Here `IsCutPartition o.1` says that `o.1` is a partition of the vertices of `g` into at least two blocks.

This statement is heavy as well, with its five class hypotheses, its three hypotheses on the input and the conjunction it states.

The second theorem bounds the probability of being right: a lower bound on the probability of success, or an upper bound on the probability of error. For Karger it takes the following form: a lower bound on the probability of obtaining a minimum cut, that is, of being exactly right.

Theorem 6.5 (Karger finds a minimum cut)**.** Under the same hypotheses, with n the number of vertices, a run of Karger returns a partition each of whose blocks is a cut of value the minimum cut value, with probability at least $2/(n(n-1))$.

In Lean:

IsMinCutOutput and karger_finds_min

```
structure IsMinCutOutput (g : MultiGraph α) (o : Finset (Finset α) × ℕ) : Prop where
  partition : g.IsCutPartition o.1
  sides : ∀ S ∈ o.1, g.IsMinCut S

theorem karger_finds_min
    {M} [Monad M] [LawfulMonad M] [inst : LawfulRandMonad M]
    [MonadCost ℕ M] [LawfulMonadCost ℕ M]
    {pick : MultiGraph α → Sym2 α → α} (hfresh : Fresh pick)
    (g : MultiGraph α) (hwf : g.WF) {n : ℕ} (hn : n = g.verts.card) (h2 : 2 ≤ n) :
    (2 : ENNReal) / ((n * (n - 1) : ℕ) : ENNReal) ≤
      ℙ_{M}[Karger pick g ∈ {o | g.IsMinCutOutput o}]
  := by ...
```

Here `IsMinCut S` says that S is a cut of value the minimum cut value.

An algorithm may mix the tiers: the treap algorithm has a support claim and an expected height, and no success probability. The lemma `eq_pure_of_support_subsingleton` is one bridge between the tiers: if every element of the support of a distribution satisfies a property that at most one value satisfies, the distribution is the point mass at that value. So a support claim plus uniqueness of the specification gives the Dirac tier, and a Las Vegas algorithm may be proved the way a Monte Carlo one is.

The support claim is also what makes amplification sound. `amplify best k m` runs m k times independently and combines the answers with the binary selector `best`; it is a program in the same monad-polymorphic style as the algorithms.

Theorem 6.6 (Amplification)**.** If one run of m lands in the success event S with probability at least p, and `best` returns a success whenever either argument is one, on the outputs V that m can produce, then k combined runs succeed with probability at least $1-(1-p)^k$.

In Lean:

amplify and amplify_success

```
def amplify {M} [Monad M] {β : Type} (best : β → β → β) : ℕ → M β → M β
  | 0, m => m
  | 1, m => m
  | k + 2, m => do
      let a ← m
      let b ← amplify best (k + 1) m
      return best a b
```

```
theorem amplify_success
    {M} [Monad M] [LawfulMonad M] [inst : LawfulRandMonad M] {β : Type}
    {best : β → β → β} {m : M β} {S V : Set β} {p : ENNReal}
    (hsupp : (𝒟[m]).support ⊆ V)
    (hclosed : ∀ a ∈ V, ∀ b ∈ V, best a b ∈ V)
    (hkeep : ∀ a ∈ V, ∀ b ∈ V, a ∈ S ∨ b ∈ S → best a b ∈ S)
    (hp : p ≤ ℙ[m ∈ S]) (k : ℕ) :
    1 - (1 - p) ^ k ≤ ℙ[amplify best k m ∈ S]
  := by ...
```

The hypotheses `hsupp` and `hclosed` restrict the success-keeping property to the reachable outputs V, which is exactly what a support claim supplies.

6.2 The three cost statements

A cost statement is about the time-marginal of Section 4, the distribution of the cost, or about a quantity computed from it. We state three, from the strongest to the weakest: the distribution of the cost determines its expectation, and the expectation gives a tail bound.

The first cost statement is the distribution of the cost itself, `costPMF` m, an element of `PMF` $\mathbb{N}$. Four case studies state it, and in each it is a point mass: reservoir sampling at $|L| - 1$, Freivalds' test on $n \times n$ matrices at $3n^2$, Schwartz–Zippel at 1 and the Fisher–Yates shuffle at 0 (Section 8). For reservoir sampling it reads as follows.

Theorem 6.7 (Cost of reservoir sampling)**.** For every lawful random monad M and every list L, the distribution of the cost of `reservoir L` is the point mass at $|L| - 1$: every run makes exactly $|L| - 1$ draws.

In Lean:

reservoir_costPMF

```
theorem reservoir_costPMF
    {M} [Monad M] [LawfulMonad M] [inst : LawfulRandMonad M]
    (L : List α) :
    costPMF (reservoir L : TimeMT ℕ M (Option α)) =
      PMF.pure (L.length - 1)
  := by ...
```

The second cost statement is the expected cost $\mathbb{E}[\text{cost } m]$, an element of $[0, \infty]$. Section 4 defined it on the joint distribution p as $\sum'_r p(r)\,\text{time}(r)$. The framework also has a more general expectation, `expVal` $p\ g = \sum'_a p(a)\,g(a)$, of any function g of the output under any distribution p, and the expected cost is `expVal` p $(r \mapsto \text{time}(r))$ by definition, with the projection $r \mapsto \text{time}(r)$ of Section 4 (`expected_cost_eq_expVal`). The general form exists because expectations of functions of the output also occur, such as the expected height of the treap (Section 8), and because Markov's inequality is proved once for `expVal` and then read at that projection. The framework also supplies the step of a recurrence: `expected_cost_uniform_step` rewrites the expected cost of a uniform draw followed by branches into the uniform average of the branch costs, $E = \frac{1}{n}\sum_i E_i$. Solving the recurrence is the mathematics and is left to the user; Quicksort in Section 8 is the example. One expected-cost statement that the framework proves once, for every algorithm, is the cost of amplification: it is linear in the number of runs, with one convention to notice. `amplify best 0 m` is by convention a single run, because a probabilistic computation cannot return no output; the statement therefore only makes sense at $k + 1$ runs.

Theorem 6.8 (Cost of amplification)**.** For every timed computation m, every selector `best` and every k, the expected cost of `amplify best (k+1) m` is $k + 1$ times the expected cost of m.

In Lean:

expected_cost_amplify

```
theorem expected_cost_amplify
    {M} [Monad M] [LawfulMonad M] [inst : LawfulRandMonad M] {β : Type}
    (best : β → β → β) (k : ℕ) (m : TimeMT ℕ M β) :
    𝔼[cost amplify best (k + 1) m] = (k + 1 : ENNReal) * 𝔼[cost m]
  := by ...
```

The third is the tail bound. It follows from the expected cost in one step, by Markov's inequality, and needs no further fact about the algorithm.

Theorem 6.9 (Markov tail bound)**.** For every timed computation m and every k, the probability that the cost of m exceeds k is at most $\mathbb{E}[\text{cost } m]/(k+1)$.

In Lean:

runtime_markov_gt

```
/-- Strict Markov tail bound: ‘P(cost > k) ≤ E[cost] / (k + 1)‘.
Since costs are ‘ℕ‘-valued this is sharper than the classical ‘E/k‘
and needs no ‘k ≠ 0‘ hypothesis. -/
theorem runtime_markov_gt
    {M : Type → Type} [Monad M] [LawfulMonad M] [LawfulRandMonad M]
    {β : Type} (m : TimeMT ℕ M β) (k : ℕ) :
    ℙ[cost m > k] ≤ 𝔼[cost m] / (k + 1 : ENNReal)
  := by ...
```

The events cost $> k$ and cost $\geq k+1$ are the same when the cost is a natural number, which is why the strict form divides by $k+1$. Any expected-cost theorem thus gives to its algorithm a tail bound in one rewrite. We also implement a second-moment form, `runtime_chebyshev`, which divides the variance of the cost by k^2, but we did not use it anywhere in our case studies.

We close with what the framework automates, stated as honestly as we can. Automation was a hard part of the work, and it is not perfect: a tactic generalises a proof pattern, and the pattern appears only after several proofs have been written. Our automation covers four tasks: the correctness of an algorithm whose output is deterministic, the expected-cost recurrence of a divide-and-conquer algorithm, the tail bound and the amplification of a success probability. The last two are one lemma each, as seen above; the first two are close to one tactic each. For instance, the proof omitted from `quicksort_correct` is the single call of the tactic `dirac_correct Quicksort`. It performs the induction that the definition induces, collapses the uniform draw, since every branch returns the same point mass, and closes each branch by rewriting with lemmas the user has tagged `@[spec_transport]` (an attribute of ours, declared in `SimpAttr.lean` of `ARA/Infrastructure/` and read by the tactics of `Correctness.lean`). For the expected cost, `cost_step` rewrites a branch into its cost, a tick adding its cost and a `pure` nothing, and hopefully leaves the recurrence, which the user solves by induction. In both cases the user provides the mathematics of the algorithm as tagged lemmas, and the hard part is to state them in the syntactic form the tactics expect: the tactics are not resilient to the many ways of stating the same mathematics, so the user has to know how they work.

The automation stops at support statements, for a reason that seems structural to us: a transport lemma (`@[spec_transport]`) is an equation and `simp` chains equations, whereas a support obligation is an implication, which `simp` cannot chain. We tried some automation for this tier as well, but we are not confident enough in it to describe it here; it can still be found in the code, in `Correctness.lean`, and Section 9 returns to the question of automation.

7 The recipe end to end: `RandMax`

This section does the analysis of `RandMax` by following almost exactly the numbered steps of the tutorial file, `Tutorial.lean` in `ARA/Algorithms/`, from its definition to the distribution of its cost, and says at each step what the user supplies and what the framework discharges. We do not quote the proofs. Most of what this section says can be found in that file, which carries the same walk with full commentary. The tutorial file, besides containing `RandMax`, contains two other algorithms that exemplify further our framework. This tutorial file will keep being updated and improved with the commits to come after this thesis, its purpose being to guide a reader to use our framework.

Step 1: the algorithm. The definition is the one quoted at the start of Section 4. The user supplies it together with its cost model, the call `MonadCost.tick 1` charging one unit per comparison.

Step 2: the readings. The readings of Section 4 are type ascriptions. Here are three of them, with a run of the two executable ones:

RandMax_IO and RandMax_IO_Timed

```
def RandMax_IO : List ℕ → IO ℕ := RandMax
#eval RandMax_IO [3, 1, 4, 1, 5, 9, 2, 6]        -- 9

noncomputable example : List ℕ → PMF ℕ := RandMax

def RandMax_IO_Timed : List ℕ → TimeMT ℕ IO ℕ := RandMax
#eval (RandMax_IO_Timed [3, 1, 4, 1, 5, 9, 2, 6]).run  -- ret 9, time 8
```

What is important to notice is that to the right of every `:=` stands the same `RandMax`; only the monad in the type on the left changes, and instance resolution does the rest. The `PMF` reading is noncomputable, so an `example` checks that the instances line up instead of running; the fourth reading, `TimeMT ℕ PMF`, is noncomputable as well.

Step 3: the branch. The user names the branch of the algorithm, `randMax_branch`: i.e. here the body of `RandMax` without its first line, with the drawn index as a parameter. It is needed for the cost proofs in the recurrence of Step 6. In Lean:

randMax_branch

```
/-- The work at a fixed index ‘i‘. -/
private abbrev randMax_branch
    (M : Type → Type) [Monad M] [RandMonad M] [MonadCost ℕ M]
    (L : List α) (i : Fin L.length) : M α := do
  MonadCost.tick 1
  let m ← RandMax (L.eraseIdx i)
  return max L[i] m
```

Step 4: the specification and its transport lemma.

Definition 7.1 (Specification of `RandMax`)**.** The specification of `RandMax` is `listMax`, the maximum of a list, with `default` (the distinguished element of the linearly ordered type α) on the empty list. Its transport lemma (`@[spec_transport]`) says that removing the element at index i and combining it by `max` with the maximum of the rest recovers the maximum of the whole list.

In Lean:

listMax and listMax_branch

```
/-- Specification: the maximum of a list (with ‘default‘ for ‘[]‘). -/
def listMax (L : List α) : α := L.foldr max default

@[spec_transport]
private lemma listMax_branch (L : List α) (i : ℕ) (h : i < L.length) :
    max L[i] (listMax (L.eraseIdx i)) = listMax L
  := by ...
```

Neither declaration mentions a monad. This is the mathematics of `RandMax` that the user must provide: the specification, and the transport lemma, which is the induction step of the correctness proof of Step 5 (its proof is three lines). The tag `@[spec_transport]` hands the equation to the tactic of Step 5.

Step 5: correctness.

Theorem 7.2 (Correctness of `RandMax`)**.** For every lawful random monad M and every list L, the output distribution of `RandMax L` is the point mass at `listMax L`: the Dirac tier of Section 6.

In Lean:

randMax_correct

```
theorem randMax_correct
    {M} [Monad M] [LawfulMonad M] [LawfulRandMonad M]
    [MonadCost ℕ M] [LawfulMonadCost ℕ M]
    (L : List α) :
    𝒟_{M}[RandMax L] = PMF.pure (listMax L)
  := by ...
```

The proof is the single call `dirac_correct RandMax` described in Section 6. This is the only correctness statement `RandMax` has. For the two other tiers and for amplification, the tutorial uses a second toy algorithm (which consists of a membership test).

Step 6: the expected cost.

Theorem 7.3 (Expected cost of `RandMax`)**.** For every lawful random monad M and every list L of length n, the expected cost of `RandMax L` is exactly n.

In Lean:

randMax_cost_exact

```
theorem randMax_cost_exact
    {M} [Monad M] [LawfulMonad M] [LawfulRandMonad M]
    (L : List α) :
    𝔼_{M}[cost RandMax L] = (L.length : ENNReal)
  := by ...
```

The notation reads `RandMax L` at `TimeMT ℕ M`, whose cost class is the `TimeMT` instance of Section 4: the ticks of `RandMax` are resolved there, and `M` itself only needs to be a lawful random monad. This is why no cost class appears among the hypotheses. The proof is a recurrence, solved by induction. The user states the recurrence in three lemmas:

- the empty list costs 0;
- the branch of Step 3 costs one unit plus the expected cost of `RandMax` on the list with the drawn element removed;
- `RandMax` on a nonempty list costs the uniform average, over the drawn index, of the branch costs.

Among these, the framework proves the first two with `cost_step`, and the third with the uniform-step lemma of Section 6. The user then solves the recurrence by induction along `RandMax.induct`, which is the induction principle that Lean derives from the definition: every branch recurses on $n-1$ elements, so by the induction hypothesis it costs $1+(n-1)=n$, and the uniform average of n copies of n is n.

Step 7: the tail bound.

Theorem 7.4 (Tail bound for `RandMax`)**.** For every lawful random monad M, every list L of length n and every k, the cost of `RandMax L` exceeds k with probability at most $n/(k+1)$.

In Lean:

randMax_cost_tail

```
theorem randMax_cost_tail
    {M} [Monad M] [LawfulMonad M] [LawfulRandMonad M]
    (L : List α) (k : ℕ) :
    ℙ[cost (RandMax L : TimeMT ℕ M α) > k] ≤ (L.length : ENNReal) / (k + 1)
  := by ...
```

The user supplies nothing new: the bound is `runtime_markov_gt` of Section 6 rewritten with `randMax_cost_exact`.

Step 8: the distribution of the cost.

Theorem 7.5 (Distribution of the cost of `RandMax`)**.** For every lawful random monad M and every list L of length n, the distribution of the cost of `RandMax L` is the point mass at n: every run makes exactly n comparisons, not only n on average.

In Lean:

randMax_costPMF

```
theorem randMax_costPMF
    {M} [Monad M] [LawfulMonad M] [inst : LawfulRandMonad M]
    (L : List α) :
    costPMF (RandMax L : TimeMT ℕ M α) = PMF.pure L.length
  := by ...
```

The user writes exactly the same induction as in Step 6 again, but with the framework's `costPMF` lemmas instead of the averaging ones. The equivalent of the uniform averaging lemma that was used for the third item of Step 6 is in this case the lemma `costPMF_lift_bind_const`, which states that if, for every value `a` that the draw may return, the branch `f a` has the same distribution of the cost `q`, then the draw followed by `f` has that distribution of the cost as well. In Lean:

costPMF_lift_bind_const

```
lemma costPMF_lift_bind_const {M} [Monad M] [LawfulMonad M]
    [inst : LawfulRandMonad M] {α β : Type}
    (m : M α) (f : α → TimeMT ℕ M β) {q : PMF ℕ}
    (h : ∀ a, costPMF (f a) = q) :
    costPMF (TimeMT.lift m >>= f) = q
  := by ...
```

Here is how it is used. On a nonempty list L of length n, the goal is `costPMF (RandMax L)` = `PMF.pure` n, and `RandMax L` is the lifted draw of an index followed by the branch of Step 3. The lemma reduces this goal to one goal for an arbitrary index i. So we need to prove that the `costPMF` of the branch at i is `PMF.pure` n. The branch at i is a tick of one unit, then `RandMax` on the list of length $n-1$ with the element at i removed, then a `return`. A tick shifts the distribution of the cost by one unit (`costPMF_tick_bind`), a `return` leaves it unchanged (`costPMF_bind_pure`), and the induction hypothesis gives `PMF.pure` $(n-1)$ for the recursive call; shifted by one unit, this is `PMF.pure` n.

Remark 7.6. It is clear that once we have this distribution, we could have derived Step 6, and hence Step 7, directly from it in one shot, but we wanted to exemplify as much as possible what our framework has developed.

8 Case studies

We tested the framework on nine algorithms. Together they cover the three correctness tiers and the three cost statements of Section 6, and they are the evidence for the claims of Section 1. Table 1 classifies them. The proofs and the statements are in the repository, one folder per algorithm, each with a Markdown file that states the same theorems in ordinary mathematical prose (written with an AI assistant, see Section 10). Two case studies are more detailed here: Quicksort, in which we exercise the cost side of the framework, and Karger's minimum cut, in which we exercise the correctness side. The other seven will be briefly explained.

8.1 Quicksort

Recall the program text of `Quicksort` from Section 5. The correctness of `Quicksort` is in the Dirac tier, and its theorem is `quicksort_correct`, quoted in Section 6. All three cost statements are proved, generically in a lawful random monad M. The first is an upper bound that holds for every list, where duplicates are allowed: the expected cost of `Quicksort L` on a list of length n is at most $\binom{n}{2}$ (`quicksort_cost_le`). This bound is attained on a list whose elements are all equal (but this attainment was not proved). The second is the exact formula, which needs distinct entries.

Theorem 8.1 (Exact expected cost of Quicksort)**.** For every lawful random monad M and every list L of n distinct elements, the expected cost of `Quicksort L` is exactly $2(n+1)H_n - 4n$, where H_n is the n-th harmonic number.

In Lean:

quicksort_cost_exact

```
/-- Exact expected complexity of Quicksort. Sorting a list of ‘n‘
distinct elements costs exactly ‘2(n+1)H(n) − 4n‘ comparisons in
expectation. -/
theorem quicksort_cost_exact
    {M} [Monad M] [LawfulMonad M]
    [inst : LawfulRandMonad M]
    (L : List α) (hnd : L.Nodup) :
    𝔼𝕣_{M}[cost Quicksort L] =
    (expected_qs_cost L.length : ℚ)
  := by ...
```

Here `expected_qs_cost n` is the rational $2(n+1)H_n - 4n$, cast into $\mathbb{R}$, and $\mathbb{ER}_M[\text{cost } e]$ is $\mathbb{E}_M[\text{cost } e]$ descended to $\mathbb{R}$ through `toReal` (using the quadratic bound proved above, which gives the finiteness that `toReal` needs). The proof follows the textbook argument. The third statement is the tail bound, which is Markov's inequality applied to the quadratic bound: for every list of length n and every k, a run of Quicksort on it exceeds k comparisons with probability at most $\binom{n}{2}/(k+1)$ (`quicksort_runtime_tail`).

8.2 Karger's minimum cut

Karger's algorithm [Kar93] was briefly described in Section 6. Its input is a multigraph $G = (V, E)$. To formalise it we first had to write some graph theory as helpers (the definition of a multigraph, cuts, contraction), part of which was taken directly from, or inspired by, GraphLib [Yin25], the Lean graph library of Sorrachai Yingchareonthawornchai.

We encountered three decisive design decisions, which were necessary to formalise Karger's algorithm as naturally as possible. The first decision was the way we formalised the set of edges E. We formalise it in Lean as a list for two reasons. One is that a uniform draw from a `Finset` exists as a distribution but not as a program (see Section 4), whereas a uniform position in the list can be drawn. The other is the multiplicity itself: formalising it in a `Finset` would have required a count attached to each edge to record its multiplicity (or something of the kind), while a list can easily have repetition.

In Lean:

MultiGraph and WF

```
structure MultiGraph (α : Type) where
  /-- The finite set of vertices. -/
  verts : Finset α
  /-- The edge list; each parallel edge is listed once per copy. -/
  edges : List (Sym2 α)

structure WF (g : MultiGraph α) : Prop where
  /-- Every endpoint of an edge is a vertex. -/
  incidence : ∀ e ∈ g.edges, ∀ v ∈ e, v ∈ g.verts
  /-- No edge is a loop. -/
  loopless : ∀ e ∈ g.edges, ¬ e.IsDiag
```

The second design is about contraction and the choice of the new vertex. A contraction replaces the two endpoints of the chosen edge by one vertex, redirects every edge through that vertex and drops the copies of the chosen edge, which have become loops. That one vertex must be of the type α of the vertices, since the recursion in Karger needs the same multigraph type, and this one vertex must be chosen. At first sight, we thought of making this choice in one of the following ways: pick the smaller endpoint (which assumes a linear order on the vertices), pick the endpoint of smaller label under an injective labelling by natural numbers fixed in advance, pick a brand-new vertex not yet in the graph (which needs the vertex type to supply new names, for instance the successor of the largest vertex on $\mathbb{N}$), or take as new vertex the union of the two endpoints (which requires the vertices to be sets: supervertices). All these ways seemed to us arbitrary (for example, why would the vertex type have a linear order?), or to rest on tools that are not the way one would have thought of contracting a graph along an edge (supervertices and their union are not the first thing one would think of). After this brainstorming we decided to assume that this choice was in fact a given, and that whoever wants to run Karger has to provide such a way of choosing. That is, we abstracted over

a function `pick` : `MultiGraph` $\alpha \to$ `Sym2` $\alpha \to \alpha$ which, given the current multigraph and the drawn edge, names a vertex. So the algorithm takes `pick` as a parameter, and the theorems additionally assume of it `Fresh pick`: the fact that the picked vertex is never an already present vertex that is not an endpoint of the drawn edge (so it is either not present at all in the graph, or one of the two endpoints). Without this hypothesis a contraction could lose two vertices instead of one, and the analysis relies on losing exactly one per round.

In Lean:

contractAt and Fresh

```
/-- Contract the ‘i‘-th listed edge into a vertex the pick
function chooses. -/
def contractAt (pick : MultiGraph α → Sym2 α → α)
    (g : MultiGraph α) (i : Fin g.edges.length) : MultiGraph α :=
  g.contractEdgeTo g.edges[(i : ℕ)] (pick g g.edges[(i : ℕ)])

/-- Freshness of a pick function: the merged vertex collides with no
untouched vertex. -/
def Fresh (pick : MultiGraph α → Sym2 α → α) : Prop :=
  ∀ {{g : MultiGraph α}} {{e : Sym2 α}}, g.WF → e ∈ g.edges →
    pick g e ∉ g.verts.filter (· ∉ e)
```

The third design was how to record the sets of vertices correctly, so that we could output the cuts. This is done through a map, carried by the loop beside the multigraph and updated at each step, that sends each live vertex to the set of original vertices merged into it. It starts by sending each vertex a to its singleton $\{a\}$, and when an edge is contracted into a vertex, we update the function only at this vertex by sending it to the union of the images of the two endpoints. At the end, the image of the set of surviving vertices under the function is a partition of the vertices, and each set in it is a cut with the same value: the number of surviving edges.

In Lean:

repOf, updateRep, contractPick and Karger

```
def repOf (rep : α → Finset α) : Sym2 α → Finset α :=
  Sym2.lift ⟨fun u v => rep u ∪ rep v, fun _ _ => Finset.union_comm _ _⟩

def updateRep (rep : α → Finset α) (e : Sym2 α) (w : α) : α → Finset α :=
  fun x => if x = w then repOf rep e else rep x

def contractPick {M} [Monad M] [RandMonad M] [MonadCost ℕ M]
    (pick : MultiGraph α → Sym2 α → α) (t : ℕ)
    (g : MultiGraph α) (rep : α → Finset α) :
    M (MultiGraph α × (α → Finset α)) :=
  if h : t + 1 ≤ g.verts.card ∧ 0 < g.edges.length then do
    -- costs one tick per edge: contracting walks the whole
    -- list to redirect it and drop the loops
    MonadCost.tick g.edges.length
    let i ← randIdx g.edges h.2
    contractPick pick t (contractAt pick g i)
      (updateRep rep g.edges[(i : ℕ)] (pick g g.edges[(i : ℕ)]))
  else
    pure (g, rep)
termination_by g.edges.length
decreasing_by exact length_edges_contractAt_lt pick g i

def Karger {M} [Monad M] [RandMonad M] [MonadCost ℕ M]
    (pick : MultiGraph α → Sym2 α → α) (g : MultiGraph α) :
    M (Finset (Finset α) × ℕ) := do
    -- t = 2 and the report function is a => {a}
  let p ← contractPick pick 2 g (fun a => {a})
  pure (p.1.verts.image p.2, p.1.edges.length)
```

The loop `contractPick` stops at `t` vertices (`t = 2` for Karger; the parameter is there for future work, Karger–Stein) or when the edges run out. In `Karger`, `p.1.verts.image p.2` is this image of the surviving vertices under the final map, and `p.1.edges.length` the number of surviving edges.

Karger's two correctness theorems are the ones quoted in Section 6. Regarding the cost, each round charges the current number of edges, and since a fresh pick loses exactly one vertex per round, a

run on a well-formed graph costs at most $(n-2)\,m$ in expectation (`karger_cost_le`). Keeping the best of k runs reports the minimum cut value with probability at least $1-(1-2/(n(n-1)))^k$ (`karger_amplified`), which is one application of the amplification theorem of Section 6.

8.3 Seven other case studies

Quickselect. Quickselect finds the k-th smallest element of a list of length n (counting from 0) without sorting the list. Like Quicksort it draws a uniform pivot and partitions the rest around it, but it then recurses on the one side that contains the wanted element, or stops when the pivot is that element. Its correctness is in the Dirac tier: for every list L and every k, the output distribution of `Quickselect L k` is the point mass at `orderStat L k`, the k-th element of the sorted list `sortSpec L` (`quickselect_correct`). Three cost statements are proved: the expected cost is at most $\binom{n}{2}$ on every list (`quickselect_cost_le_quadratic`), at most $4n$ on a list with distinct entries (`quickselect_cost_le_linear`); and, on a list with distinct entries and for $k < n$, the expected cost is exactly Knuth's formula [Knu71] (`quickselect_cost_exact`):

$$2\big(n+3+(n+1)H_n-(k+3)H_{k+1}-(n+2-k)H_{n-k}\big).$$

Reservoir sampling. Reservoir sampling, Algorithm R of Vitter [Vit85] with a reservoir of one element, draws a uniformly random element of a list in a single pass, without knowing the length of the list in advance. Section 6 described it and quoted its two theorems: `reservoir_correct`, in the exact-distribution tier, gives the output distribution, and `reservoir_costPMF` gives the distribution of the cost, the point mass at $n-1$. We also give one corollary: on a list with distinct entries, every element is returned with probability exactly $1/n$ (`reservoir_correct_of_nodup`).

Fisher–Yates. The Fisher–Yates shuffle produces a uniformly random permutation of a list. Our version moves a uniformly chosen element to the front and recurses on the rest. We made a support claim: every output is a permutation of the input (`support_shuffle`). For a list with distinct entries the correctness statement of the algorithm lies in the exact-distribution tier: the output distribution is uniform over the $n!$ permutations of the input (`shuffle_uniform`), each returned with probability $1/n!$ (`shuffle_perm_apply`). The shuffle is treated as a sampler for other algorithms (the treap algorithm below uses it), like the primitive draws of Section 4. So we deliberately did not take the cost interface, and there are no ticks. We could have charged one tick per draw, and the distribution of the cost would then be the point mass at n.

Freivalds. Freivalds' verifier [Fre77] tests whether $AB = C$ for three $n \times n$ matrices without computing the product AB: as Section 5 described, it draws a uniform 0/1 vector r and compares $A(Br)$ with Cr. We formalise it over a commutative ring. Its correctness statement is in the support-plus-probability tier: equal products are always accepted (`freivalds_complete`), and unequal products are accepted with probability at most $1/2$ (`freivalds_sound`). The three matrix–vector products are charged as one tick of $3n^2$, so the distribution of the cost is the point mass at $3n^2$ (`freivalds_costPMF`).

Schwartz–Zippel. The Schwartz–Zippel tester [Sch80, Zip79] decides whether a polynomial P in n variables over an integral domain is the zero polynomial, without expanding it (hence its utility). It evaluates P at a uniformly random point of the grid S^n, for a finite set S of elements of the domain, and accepts when the value is 0. Its correctness statement is in the support-plus-probability tier: the zero polynomial is always accepted (`schwartzZippel_complete`), and a nonzero polynomial of total degree d is accepted with probability at most $d/|S|$ (`schwartzZippel_sound`). The evaluation is charged as one tick, so the distribution of the cost is the point mass at 1 (`schwartzZippel_costPMF`).

Treap. A treap is a binary search tree on n distinct keys whose shape is made random, by inserting the keys in a uniformly random order, so that its height stays logarithmic in expectation. We wrote

two models of it. The insertion model shuffles the list of keys with the Fisher–Yates shuffle and then inserts the keys one by one, in the order of the shuffled list, into an initially empty binary search tree, with the usual insertion. The other model, which is recursive, draws a uniformly random key of the list as the root, then recurses on the keys smaller than the root to build the left subtree and on the keys larger than the root to build the right subtree. Their correctness statement is a support claim: every tree they can output is a valid binary search tree over the keys (`randomBST_correct`, `treap_correct`). The fact that the two models have the same output distribution is future work. We prove for the recursive model that the expected height is at most $3\lfloor \log_2(n+3) \rfloor + 4$ (`treap_expected_height_le`); the constant 3 in front of the logarithm is close to the sharp one, since the expected height is asymptotically $4.311 \ln n \approx 2.99 \log_2 n$ [Dev86]. Neither model takes the cost interface.

Coupon collector. The coupon collector draws coupons uniformly among n objects, independently, until every object has been seen; the question is how many draws this takes. It is the one case study without a program: its loop, draw until a new object appears, terminates only almost surely. So it is not a structurally terminating Lean program (Section 9 returns to such loops). We still wrote its cost distribution directly in `PMF`, as follows. When m of the n objects are still missing, each draw shows a new object with probability m/n, so the number of draws until the next new object is a geometric distribution with this success probability m/n, which the framework provides. We call this a stage (it goes from $n-m$ to $n-m+1$ objects found). The distribution of the total number of draws is then written with one `bind` per stage. That is, draw the number of draws of the first stage, then draw, independently, the total of the remaining stages, and return their sum. Under this way of formulating the "cost distribution" of the coupon collector, we prove that the expected number of draws is exactly $\sum_{r=0}^{n-1} n/(r+1)$ (`couponCollector_cost_exact`), that is nH_n in $\mathbb{R}$ (`couponCollector_cost_exact_real`).

Case study	Correctness tier	Cost statements
Quicksort	Dirac	exact $2(n+1)H_n - 4n$; bound $\binom{n}{2}$; tail bound
Quickselect	Dirac	exact (Knuth); bounds $4n$, $\binom{n}{2}$
Karger	support plus probability	expected cost $\leq (n-2)m$
Reservoir sampling	exact distribution	point mass at $n-1$
Fisher–Yates	exact distribution	charges no cost
Freivalds	support plus probability	point mass at $3n^2$
Schwartz–Zippel	support plus probability	point mass at 1
Treap	support claim	none (expected height instead)
Coupon collector	distribution study	exact nH_n

Table 1: The nine case studies. The middle column uses the tier names of Section 6 where a tier applies; the treap has a support claim only, and the coupon collector has no program layer. The right column names the cost statements that we proved (the treap has none; its expected height stands in their place); n is the size of the input, a list length or a number of vertices, and m the number of edges.

9 Limitations and future work

The principal limitation was stated in Section 5. Nothing connects a tick call to the work the surrounding code performs. Every cost theorem is therefore a theorem about the cost model as written, and the reader must trust where the ticks were placed and what they charge. The direction that could remove this limitation seems clear, though we have not taken it: once the semantics of a program is itself formalised and executable, a cost model attached to that semantics is checked by the kernel rather than trusted. Work of this kind exists in the Lean ecosystem. Loom builds foundational program verifiers on a monadic shallow embedding of an executable semantics; LeanCP does foundational verification of C. We report these as directions only, and we have not evaluated them.

The second limitation is the restriction to total mass one. A `PMF` sums to 1, so it cannot describe a program that may run forever, and a loop that retries until it succeeds, such as the coupon collector of Section 8, is not a `PMF` program. The classical solution for this problem is a sub-probability distribution, whose mass may be less than 1, the missing mass being the probability that the program never stops. Our component `SPMF` implements it as `OptionT` over `PMF`. This is a distribution over `Option` α, where the probability of `none` is the probability of running forever and the mass of the program is what remains. On it we define `retry good p`, the program that runs `p` again and again until its output satisfies the test `good`. We do not define it as a loop, which Lean would reject since it may not terminate, but directly by its distribution. It is the distribution of one attempt conditioned on the event "the attempt succeeds or runs forever". A failed attempt is simply run again, so it does not change the odds between the outcomes that end the loop. We proved a Las Vegas theorem, `mass_retry_eq_one`, which simply states that if one attempt always stops and succeeds with positive probability, then `retry` will stop with probability one. `SPMF` is not yet connected to the correctness or cost machinery of Sections 4 and 6. It is kept apart, in the folder `ARA/FutureWork/` of the repository.

Third, every distribution in this thesis is discrete. Continuous distributions would replace `PMF` by `MeasureTheory.Measure`, and would carry sigma-algebras and measurability side conditions through every layer of the framework. This is much harder to implement, and so we judged it out of scope.

Fourth, the probability file (`Prob.lean`) defines the probability of an event and the expectation of a function of the output, and nothing conditional. A conditional probability and a conditional expectation would probably be definitions of the same kind, and would let the analyses that argue by conditioning on an event be stated as they are on paper. Another addition would be to descend everything to $\mathbb{R}$ automatically. The type `ENNReal` is unpleasant to work with (there is no ring tactic for it, for instance), and it serves only to make the definitions well defined, through the `tsum` of Section 3.

Fifth, the concentration bounds (on the cost) stop at the Chebyshev ones. No other concentration bound is provided, and nothing sharper is available yet in the framework.

Sixth, the automation of Section 6 stops at the support tier, and further automation will need more patterns of the kind we found for the Dirac tier and the expected cost. Finding the right level of abstraction for such patterns is difficult and future work.

Seventh, no external user has yet been through the tutorial. Our own use of the framework was with an AI assistant (Section 10), and what an AI assistant understands is not what a newcomer to this monadic setup understands. So a piece of future work is better documentation, and the care of the repository: cleaning it, shortening proofs, making things more natural, and maintaining it.

The eighth and final limitation concerns the speed of the executable reading. Our algorithms are written to be convenient for proof, not for speed, and we do not know whether an executable reading written for speed can be made. But fast algorithms would improve the playground of experimental mathematics and of agentic AI, because an executable text lets an automated agent test and compute with the very text it reasons about. We ourselves ran loops in which our assistant had to write an algorithm, run it at `IO` on small inputs to see its output and its tick count, and then prove. A framework in which the same text runs and is proved keeps that loop short. As assistants write more of the code, we expect this to matter more, but it is an expectation and we have not measured it. Moreover we do not claim that Lean could take the place of a computer algebra system such as Magma or PARI/GP, whose implementations are optimised in C or C++, down to the machine code in places. Still, the authors of Lean report that Lean code compiled to machine code, through C, is competitive with established functional compilers [UM19]. This leaves room, if we are optimistic, for trying to implement fast algorithms and, who knows, to tie our framework to them.

With these limits stated, we conclude with some confidence that a large class of randomised algorithms, those that are guaranteed to terminate, can be written once. Their correctness and their cost are obtained as two readings of the same text, at the price of trusting the cost model written into the text, that is, where the tick calls were placed and what they charge.

10 Use of generative AI

We used a generative AI assistant throughout this work, as the declaration of originality at the end of this document records. From the beginning of the thesis (8 March 2026) to its end (23 August 2026), the tool used was Claude Code (Anthropic), with the Opus and Fable models.

The assistant wrote code, and we directed it. The design of the framework, its classes, its readings and its notions, is ours, in combination with discussions with Sorrachai Yingchareonthawornchai. Still, their Lean code was mainly written by the models and refined with them. The choice of the nine algorithms to study was partly made by the assistant and partly by us: we wanted reservoir sampling, Quicksort, Quickselect and Karger in any case, and the assistant's choice of the other algorithms was still guided by our taste and our needs. The Lean proofs, the tactics, the tutorial file and the companion files of the case studies were also partly written by the assistant, from our instructions and in many rounds of correction. The design decisions for Karger were ours. The text of this thesis was a draft of ours, refined with the assistant, which corrected its typos and its syntax, all this in a loop of discussion: the assistant improving our text and we improving its output and taking the decisions. The companion files of the case studies, which state each algorithm and its theorems in English, were written by the assistant, and we did not check them with the care we gave to the Lean.

References

[ANS21] Reynald Affeldt, David Nowak, and Takafumi Saikawa. A trustful monad for axiomatic reasoning with probability and nondeterminism. *Journal of Functional Programming*, 31:e17, 2021.

[BCM+26] Clark Barrett, Swarat Chaudhuri, Fabrizio Montesi, Jim Grundy, Pushmeet Kohli, Leonardo de Moura, Alexandre Rademaker, and Sorrachai Yingchareonthawornchai. CSLib: The Lean computer science library, 2026. arXiv:2602.04846. Software available at `https://github.com/leanprover/cslib`.

[Chr23] David Thrane Christiansen. *Functional Programming in Lean*, 2023. Online book, `https://lean-lang.org/functional_programming_in_lean/`.

[dMU21] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.

[Dev86] Luc Devroye. A note on the height of binary search trees. *Journal of the ACM*, 33(3):489–498, 1986.

[EHN20] Manuel Eberl, Max W. Haslbeck, and Tobias Nipkow. Verified analysis of random binary tree structures. *Journal of Automated Reasoning*, 64(5):879–910, 2020.

[Fre77] Rūsiņš Freivalds. Probabilistic machines can use less running time. In *Information Processing 77, Proceedings of IFIP Congress 77*, pages 839–842. North-Holland, 1977.

[Höl17] Johannes Hölzl. Markov chains and Markov decision processes in Isabelle/HOL. *Journal of Automated Reasoning*, 59(3):345–387, 2017.

[Kar93] David R. Karger. Global min-cuts in RNC, and other ramifications of a simple min-cut algorithm. In *Proceedings of the Fourth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA '93)*, pages 21–30, 1993.

[Knu71] Donald E. Knuth. Mathematical analysis of algorithms. In *Proceedings of the IFIP Congress 1971, Ljubljana*, volume 1, pages 19–27. North-Holland, 1971.

[mat20] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020.

[MR95] Rajeev Motwani and Prabhakar Raghavan. *Randomized Algorithms*. Cambridge University Press, 1995.

[PM15] Adam Petcher and Greg Morrisett. The foundational cryptography framework. In *Principles of Security and Trust (POST 2015)*, volume 9036 of *Lecture Notes in Computer Science*, pages 53–72. Springer, 2015.

[Sch80] Jacob T. Schwartz. Fast probabilistic algorithms for verification of polynomial identities. *Journal of the ACM*, 27(4):701–717, 1980.

[TH19] Joseph Tassarotti and Robert Harper. A separation logic for concurrent randomized programs. *Proceedings of the ACM on Programming Languages*, 3(POPL), Article 64, 2019.

[UM19] Sebastian Ullrich and Leonardo de Moura. Counting immutable beans: reference counting optimized for purely functional programming. In *Implementation and Application of Functional Languages (IFL '19)*, pages 3:1–3:12. ACM, 2019.

[Vit85] Jeffrey Scott Vitter. Random sampling with a reservoir. *ACM Transactions on Mathematical Software*, 11(1):37–57, 1985.

[Yin25] Sorrachai Yingchareonthawornchai. GraphLib: a Lean 4 library for graph theory and graph algorithms, 2025–2026. Software available at `https://github.com/sorrachai/GraphLib`.

[Zip79] Richard Zippel. Probabilistic algorithms for sparse polynomials. In *Symbolic and Algebraic Computation (EUROSAM '79)*, volume 72 of *Lecture Notes in Computer Science*, pages 216–226. Springer, 1979.